



I302 - Aprendizaje Automático y Aprendizaje Profundo

Trabajo Práctico 3: Redes Neuronales

Ignacio Carlos Basso

13 de mayo de 2025

Ingeniería en Inteligencia Artificial

Resumen

En este trabajo se aborda el problema de clasificación de caracteres japoneses a partir de imágenes en escala de grises de 28×28 píxeles, utilizando redes neuronales multicapa. Se implementó una red neuronal base (M0) con *backpropagation* y descenso por gradiente, y posteriormente se introdujeron mejoras como *rate scheduling*, *mini-batch SGD*, optimizador Adam, regularización L2, *early stopping* y *dropout*, evaluando su impacto mediante búsqueda de hiperparámetros tipo *grid search*.

El modelo más efectivo (M3) resultó de combinar Adam con una arquitectura de tres capas ocultas de 500, 300 y 100 neuronas. Este alcanzó un *accuracy* de 0.65 en validación y de 0.60 en test, superando a la arquitectura base en rendimiento y eficiencia. Además, se exploraron arquitecturas para inducir *overfitting* (M4) y se descartaron configuraciones con baja capacidad de generalización o dificultades de convergencia.

Finalmente, se utilizó el modelo M3 para predecir las probabilidades a posteriori del conjunto `X_COMP.npy`, generando el archivo de salida solicitado en formato `.csv`.

1. Introducción

En este trabajo se aborda el problema de clasificación de imágenes de caracteres japoneses, utilizando redes neuronales multicapa. El conjunto de datos consiste en imágenes en escala de grises de 28×28 píxeles, cada una perteneciente a una de 49 clases posibles de símbolos. Por lo tanto, la entrada del algoritmo es una imagen y la salida es una distribución de probabilidad sobre las posibles clases.

Para resolver este problema, se implementaron y compararon distintos modelos de redes neuronales, desde una arquitectura base (M0) hasta versiones mejoradas (M1), al incorporar técnicas como esquemas de ajuste de tasa de aprendizaje (*rate scheduling*), *Mini-batch stochastic gradient descent*, optimizadores avanzados (Adam), regularización L2, regularización mediante *early stopping* y regularización mediante *dropout*. Posteriormente, se replicó el modelo con la mejor configuración utilizando la librería *PyTorch*, dando lugar a los modelos M2, M3 (explorando cambios en la cantidad de capas ocultas y unidades ocultas por capa) y M4 (con una arquitectura que produzca *overfitting*).

El objetivo final es evaluar el impacto de cada mejora sobre la performance del modelo y seleccionar la mejor arquitectura en términos de generalización, medida principalmente por el error de validación. Para de esta mane-

ra, predecir las probabilidades a-posteriori de cada clase del dataset `X_COMP`.

2. Métodos

2.1. Redes neuronales multicapa (MLP)

Las redes neuronales multicapa (MLP) conforman una familia de modelos compuestos por varias capas de unidades de procesamiento, donde cada nodo en una capa está conectado a todos los de la capa siguiente. Esta estructura incluye una capa de entrada, una o más capas intermedias (ocultas) y una capa de salida. Cada neurona aplica una función de activación sobre una combinación lineal de sus entradas, ponderadas por sus respectivos pesos y con un término de sesgo.

En este trabajo, se utilizó la función de activación ReLU, definida como:

$$\text{ReLU}(x) = \max(0, x) \quad (1)$$

La activación ReLU introduce no linealidades en el modelo, lo que le permite capturar relaciones complejas en los datos. Para la capa final, se emplea una función de activación acorde al tipo de tarea; en este caso de clasificación multiclase, se utilizó la función softmax.

La función softmax transforma los valores de activación de la salida en probabilidades

normalizadas, y se define por:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \quad (2)$$

donde z_i representa la salida de la neurona i y C es la cantidad total de clases. Este mecanismo garantiza que las probabilidades sumen uno, facilitando la interpretación probabilística de la salida.

El entrenamiento del modelo se realiza mediante el algoritmo de *Backpropagation* (2.2), el cual ajusta los parámetros de la red minimizando una función de *loss*. En este estudio, se empleó la entropía cruzada multiclase, una métrica que cuantifica la diferencia entre la distribución predicha y la verdadera. Su formulación es:

$$L(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N y_i \log(\hat{y}_i) \quad (3)$$

donde $y_i \in \{0, 1\}$ representa la clase real codificada en one-hot, y $\hat{y}_i \in (0, 1)$ es la probabilidad estimada por el modelo para esa clase. Esta función penaliza fuertemente las predicciones incorrectas con alta confianza, promoviendo un aprendizaje robusto.

2.2. Backpropagation

El algoritmo de *Backpropagation* es un método eficiente para calcular los gradientes de la función de pérdida con respecto a los parámetros de la red. Se basa en la regla de la cadena y se compone de dos fases: la fase hacia adelante (*forward pass*) y la fase hacia atrás (*backward pass*). En la *forward pass*, se calcula la salida de la red y se evalúa la función de pérdida. En la *backward pass*, se calculan los gradientes de los pesos y sesgos utilizando la regla de la cadena. Estos gradientes se utilizan para actualizar los parámetros de la red mediante un optimizador, como el descenso del gradiente estocástico (SGD) o Adam.

2.3. Rate scheduling

Para mejorar la eficiencia del entrenamiento y facilitar la convergencia, se implementaron dos estrategias de ajuste dinámico del *learning rate* (*rate scheduling*): decaimiento lineal con saturación y decaimiento exponencial.

En el decaimiento lineal, el *learning rate* disminuye a una tasa constante en cada época, hasta alcanzar un valor mínimo predefinido:

$$\text{lr}_{\text{lineal}}(t) = \max(\text{lr}_0 - \lambda t, \text{lr}_{\min}) \quad (4)$$

donde lr_0 es el *learning rate* inicial, λ es la tasa de decaimiento y t es la época.

En el decaimiento exponencial, el *learning rate* decrece multiplicativamente a lo largo del tiempo:

$$\text{lr}_{\text{exp}}(t) = \text{lr}_0 \cdot e^{-\lambda t} \quad (5)$$

Ambas estrategias fueron integradas en el ciclo de entrenamiento y permiten controlar el ritmo de aprendizaje según la etapa del proceso de optimización.

2.4. Mini-batch stochastic gradient descent (SGD)

El método de descenso por gradiente estocástico con mini-batches permite actualizar los parámetros de la red usando pequeños subconjuntos aleatorios del conjunto de entrenamiento, lo que acelera la convergencia y reduce la varianza de las actualizaciones.

En cada época, los datos son aleatorizados y divididos en mini-batches de tamaño fijo. Para cada mini-batch se realiza una pasada completa de propagación hacia adelante, retropropagación y actualización de parámetros. Al finalizar cada época, se evalúa el costo total sobre el conjunto completo de entrenamiento y validación para monitorear el progreso.

Esta estrategia combina la estabilidad del descenso por gradiente batch completo con la

eficiencia del enfoque estocástico, permitiendo un entrenamiento más robusto y eficiente.

2.5. Optimizador Adam

El optimizador Adam (*Adaptive Moment Estimation*) combina las ventajas del método de *momentum* y de RMSProp. Este algoritmo ajusta el *learning rate* de forma individual para cada parámetro, utilizando promedios móviles de primer y segundo momento del gradiente, los cuales son corregidos por sesgo. La actualización de los parámetros se expresa como:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (6)$$

donde $\hat{m}_t$ y $\hat{v}_t$ representan los momentos corregidos, α es el *learning rate* y ϵ es un pequeño valor para evitar divisiones por cero.

En este trabajo, Adam fue incorporado como una opción dentro del procedimiento general de entrenamiento, y puede ser utilizado tanto con el conjunto completo de datos como con mini-batches. Los vectores de momentos se inicializan en cero y se actualizan iterativamente en cada paso del entrenamiento. Esta estrategia permitió mejorar la estabilidad numérica y acelerar la convergencia del modelo.

2.6. Regularización L2

La regularización L2 penaliza grandes valores de los pesos para evitar el *overfitting*, añadiendo el siguiente término a la función de pérdida:

$$L_{\text{reg}} = \frac{\lambda}{2} \sum_{l=1}^L \|W^{[l]}\|^2 \quad (7)$$

Durante el *Backpropagation*, este término ajusta los gradientes de cada capa. En la implementación, la penalización se escala por $\frac{\lambda}{m}$ y se suma a los gradientes de los pesos, incentivando soluciones con pesos más pequeños y mejor capacidad de generalización.

2.7. Regularización mediante dropout

La regularización mediante *dropout* es una técnica de regularización que consiste en desactivar aleatoriamente un porcentaje de neuronas durante el entrenamiento. En cada capa, se aplica una máscara binaria a las activaciones y se escala por $1/p$, donde p es la probabilidad de retención. Esto fuerza a la red a no depender de neuronas específicas, mejorando la generalización. Durante la inferencia, se usan todas las neuronas sin aplicar máscaras.

2.8. Inicialización de Parámetros

Los parámetros de la red se inicializan mediante la función `initialize_parameters`, que recorre las capas especificadas en `layer_dims`. Los pesos $W^{[l]}$ se inicializan aleatoriamente con la estrategia de He: $W^{[l]} \sim \mathcal{N}(0, \frac{2}{n[l-1]})$, y los sesgos $b^{[l]}$ se inicializan en cero. Esta inicialización busca mantener la varianza de las activaciones estable entre capas, facilitando el entrenamiento.

2.8.1. Descripción del conjunto de datos y preprocesamiento

El conjunto de datos utilizado consiste en imágenes en escala de grises de tamaño 28×28 píxeles, correspondientes a caracteres japoneses. Cada imagen pertenece a una de las 49 clases posibles. Un conjunto de ejemplos se presenta en la Figura 1, permitiendo observar que poseen patrones visuales diferentes, a pesar de que algunos caracteres son similares entre sí implicando un desafío para el modelo. Para su procesamiento, cada imagen fue vectorizada y representada como un arreglo unidimensional de 784 características.

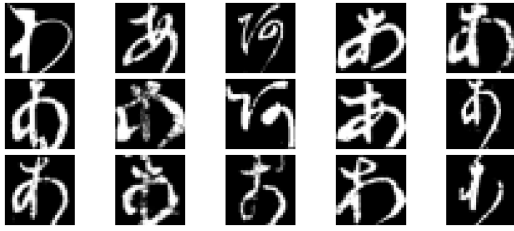


Figura 1: Ejemplo de imagen del conjunto de datos.

Se realizó una división estratificada en tres subconjuntos: 64 % para entrenamiento, 16 % para validación y 20 % para prueba. Esta división preserva la distribución original de clases en cada subconjunto.

Como preprocesamiento, los valores de los píxeles fueron normalizados dividiendo por 255, para que queden en el rango $[0, 1]$, lo que facilita la convergencia del entrenamiento. Además, las etiquetas se codificaron utilizando one-hot encoding para ser compatibles con la función de pérdida de entropía cruzada multiclase.

2.8.2. Selección de hiperparámetros

La selección de hiperparámetros se realizó mediante una búsqueda en cuadrícula (*grid search*) sobre un conjunto predefinido de valores para cada técnica de mejora. En una primera etapa, se evaluaron distintas combinaciones de hiperparámetros por separado, con el objetivo de identificar los valores óptimos para cada mejora individual. Luego, se llevó a cabo una segunda búsqueda combinando estas mejoras, utilizando sus mejores configuraciones, para determinar la arquitectura y configuración final más adecuada.

A continuación, se detallan los conjuntos de hiperparámetros evaluados inicialmente para cada técnica implementada. En todos los casos, se utilizaron distintas configuraciones de arquitectura e hiperparámetros para cada mejora (los marcados con * corresponden a la configuración óptima para cada técnica):

- **Arquitectura de la red:**

- [784, 100, 49]: una capa oculta (óptima para *Mini-batch SGD* y Adam).
- [784, 100, 80, 49]: dos capas ocultas (óptima para L2).
- [784, 200, 100, 80, 49]: tres capas ocultas (óptima para *Rate scheduling*, *Early stopping*, *Dropout*).

- **Rate scheduling:**

- Lineal: `learning_rate = 0.01, 0.005`, `epochs = 5000`.
- Exponencial*: `learning_rate = 0.01*`, `0.005`, `epochs = 5000`.

- **Mini-batch SGD:** `batch_size = 32, 64*, 128, 256`; `learning_rate = 0.01*`, `0.005`, `epochs = 200`.

- **Adam:** `learning_rate = 0.005, 0.001*, 0.0001`; `beta1 = 0.8, 0.85, 0.9*, 0.95`; `beta2 = 0.95, 0.99, 0.999*`; `epsilon = 10-6, 10-7, 10-8*`; `epochs = 100`.

- **L2:** `lambd = 0.01, 0.001*`; `learning_rate = 0.01*, 0.005`; `epochs = 5000`.

- **Early stopping:** `patience = 10, 20, 100*`; `learning_rate = 0.01*, 0.005`, `epochs = 5000`.

- **Dropout:** `keep_prob = 0.6, 0.8*`; `learning_rate = 0.01*, 0.005`; `epochs = 5000`.

Estas configuraciones fueron evaluadas utilizando el conjunto de validación y permitieron identificar los hiperparámetros óptimos de cada técnica en forma individual. En una segunda etapa, se realizó una búsqueda combinada utilizando dichos valores óptimos como base, con el objetivo de encontrar la mejor configuración general del modelo.

Dado que el optimizador Adam fue el que mostró mejor rendimiento durante la primera etapa, se decidió fijar sus parámetros, así como limitar el número de épocas a 100. Esta

elección se basó en un compromiso entre evitar el *overfitting* y mantener un buen desempeño, además de reducir el espacio de búsqueda por razones computacionales.

A partir de esta búsqueda combinada, se evaluaron distintas configuraciones integrando varias mejoras. Se presentan algunas de las combinaciones más relevantes en el Cuadro 4. Todas las combinaciones exploradas se encuentran detalladas en la Sección 3 del notebook, donde se documenta el proceso completo de búsqueda de hiperparámetros y entrenamiento.

2.9. Métricas de performance

Las métricas empleadas para evaluar el desempeño de los modelos incluyen la *Cross-Entropy*, la *matriz de confusión* y la *accuracy*.

La *Cross-Entropy* cuantifica la discrepancia entre la distribución de probabilidades predicha por el modelo y la distribución real (etiquetas verdaderas), siendo especialmente adecuada para tareas de clasificación multiclase. Su expresión es la siguiente:

$$\text{Cross-Entropy}(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^N y_i \log(\hat{y}_i) \quad (8)$$

donde N representa el número total de muestras, $y_i \in \{0, 1\}$ es el valor real de la clase en formato one-hot, y $\hat{y}_i \in (0, 1)$ es la probabilidad estimada por el modelo para la clase correspondiente.

La *matriz de confusión* ofrece una representación visual de los aciertos y errores cometidos por el modelo, comparando las clases predichas con las verdaderas. Es una herramienta útil para detectar desequilibrios o patrones específicos de error entre clases.

Por último, la *accuracy* (precisión) mide la proporción de predicciones correctas respecto al total de predicciones realizadas. Se define como:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (9)$$

donde TP, TN, FP y FN representan los verdaderos positivos, verdaderos negativos, falsos positivos y falsos negativos, respectivamente.

3. Resultados

En el punto 2, se implementó una red neuronal multicapa con dos capas ocultas, compuestas por 100 y 80 neuronas respectivamente, utilizando la función de activación ReLU en las capas ocultas y softmax en la capa de salida.

Este modelo se denomina **M0** y sirve como *baseline* para las mejoras posteriores. A continuación, se presentan los resultados obtenidos en términos de precisión (*accuracy*), función de pérdida (cross-entropy), matriz de confusión y tiempo de ejecución, para el conjunto de validación. Además, se incluye la evolución del valor de la función de costo a lo largo de las épocas.

Val. Acc.	Val. Loss	Tiempo (min)
0.59	1.57	23

Cuadro 1: Resultados del modelo base (M0) en términos de rendimiento y tiempo.

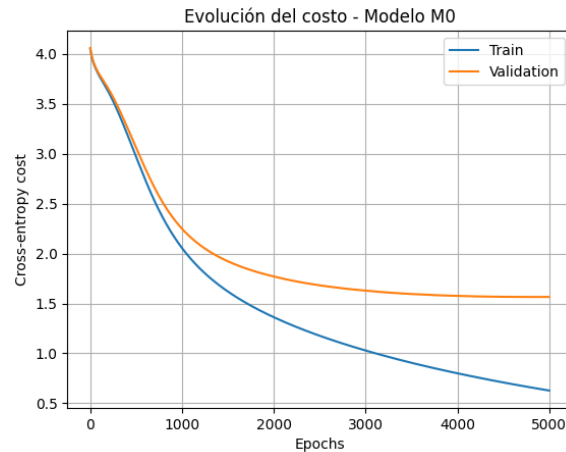


Figura 2: Evolución de la función de *loss* durante el entrenamiento del modelo (M0).

Como se observa en la Figura 2, el costo

disminuye de manera significativa tanto en el conjunto de entrenamiento como en el de validación durante las primeras épocas. Sin embargo, a partir de aproximadamente la época 1000, la reducción del costo comienza a desacelerarse, y eventualmente se estabiliza.

Un aspecto importante a considerar es la brecha entre las curvas de entrenamiento y validación. Si bien el modelo continúa perfeccionando su rendimiento sobre el conjunto de entrenamiento, la mejora en validación se estabiliza en etapas tempranas del proceso. Esta divergencia podría interpretarse como un indicio incipiente de sobreajuste; sin embargo, no resulta concluyente, ya que la diferencia entre ambas métricas no es excesiva. En concreto, se observa un *accuracy* de entrenamiento de 0.86 y una *loss* de 0.63, frente a un *accuracy* de validación de 0.59 y una *loss* de 1.57 (ver Tabla 1). Estas discrepancias moderadas sugieren que el modelo conserva una capacidad de generalización aceptable.

No obstante, los resultados también indican que la arquitectura base del modelo M0 podría beneficiarse de mejoras adicionales. La incorporación de técnicas de regularización, como *L2*, *early stopping* o *dropout*, así como posibles ajustes en la arquitectura, mejorando así la capacidad del modelo para generalizar a datos no vistos.

Consecuentemente, en el punto 3, se implementaron diversas mejoras sobre el modelo base, entre las cuales se encuentran *rate scheduling* (Sección 2.3), descenso por gradiente estocástico con mini-batches (Sección 2.4), optimizador Adam (Sección 2.5), regularización L2 (Sección 2.6) y regularización mediante *dropout* (Sección 2.7).

Para el entrenamiento de los modelos, se llevó a cabo una búsqueda de hiperparámetros mediante *grid search*, tal como se detalla en la Sección 2.8.2. En una primera etapa, se exploraron los hiperparámetros óptimos para cada técnica por separado. Posteriormente, se realizó una segunda búsqueda combinando las mejores configuraciones obtenidas, con el

fin de identificar la arquitectura y conjunto de mejoras que ofrecieran el mejor desempeño general.

El Cuadro 5 resume algunas de las combinaciones más representativas evaluadas en esta segunda etapa. Del análisis de los resultados se observa que varios modelos alcanzan valores similares de *accuracy* sobre el conjunto de validación (alrededor de 0.60–0.62). Sin embargo, el modelo que combinó la arquitectura [784, 100, 49] con el optimizador Adam ($\text{lr}=0.001$, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon = 10^{-8}$) se destacó por lograr el menor valor de *loss* de validación (1.52) con un tiempo de entrenamiento reducido (0.13 minutos).

Otros modelos, como los que incorporaron Adam junto con *rate scheduling* exponencial o mini-batches, presentaron desempeños similares en términos de *accuracy*, aunque con un leve aumento en la *loss* y el tiempo de entrenamiento. En particular, los modelos con arquitecturas más profundas (por ejemplo, [784, 200, 100, 80, 49]) no lograron mejoras significativas en las métricas de validación, a pesar de obtener una menor pérdida en entrenamiento, lo que sugiere un mayor riesgo de *overfitting*.

En base a estos resultados, se seleccionó como configuración final **M1** aquella basada en Adam con arquitectura reducida, por su buen compromiso entre precisión, pérdida y eficiencia computacional. En la Figura 3 se presenta la evolución de la función de costo durante el entrenamiento del modelo M1, que muestra una tendencia similar a la del modelo base, pero con una convergencia más rápida y estable.

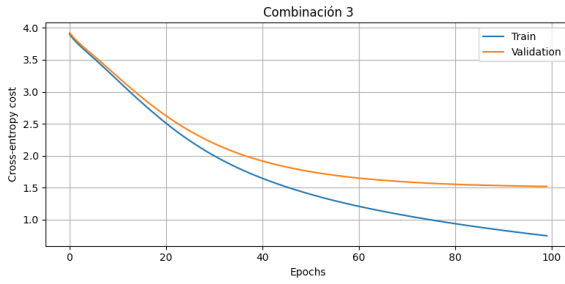


Figura 3: Evolución de la función de *loss* durante el entrenamiento del modelo M1.

Una vez obtenido el modelo M1, utilizando *PyTorch* se entrenó una red neuronal con su arquitectura e hiperparámetros, llamando a este modelo **M2**. En el Cuadro 2 se presentan los resultados obtenidos por el modelo M2. Se observan leves diferencias numéricas respecto a la implementación del modelo M1, atribuibles a la aleatoriedad inherente en la inicialización de los pesos y en el proceso de entrenamiento, así como también a diferencias en la gestión de la optimización por parte de *PyTorch*.

Val. Acc.	Val. Loss	Tiempo (min)
0.61	1.55	0.06

Cuadro 2: Resultados del modelo M2 en términos de rendimiento y tiempo.

Consignientemente, se exploraron cambios en la cantidad de capas ocultas y unidades ocultas por capa, con el objetivo de determinar la mejor configuración posible. Se evaluaron arquitecturas con entre 1 y 4 capas ocultas, utilizando distintas cantidades de neuronas por capa. En total, se probaron 6 arquitecturas distintas, todas con una capa de entrada de tamaño 784 y una capa de salida de 49 neuronas, correspondiente a la cantidad de clases del problema. Las configuraciones evaluadas fueron las siguientes:

- [784, 100, 49]: una capa oculta con 100 neuronas.

- [784, 200, 100, 49]: dos capas ocultas con 200 y 100 neuronas.
- [784, 300, 150, 50, 49]: tres capas ocultas con tamaños decrecientes.
- [784, 400, 200, 100, 50, 49]: cuatro capas ocultas con reducción progresiva.
- [784, 500, 300, 100, 49]: tres capas ocultas con distribución asimétrica.
- [784, 200, 200, 200, 49]: tres capas ocultas con el mismo tamaño.

Estas variantes permitieron evaluar el impacto de la profundidad y del ancho de la red sobre el rendimiento final del modelo. Los desempeños obtenidos se presentan en el Cuadro 6.

Se observa que la arquitectura con tres capas ocultas de 500, 300 y 100 unidades fue la que obtuvo el mejor desempeño, alcanzando un *accuracy* de validación de 0.65 y la menor *loss* (1.41). Esta configuración se destaca por su mayor capacidad de representación, sin incurrir en *overfitting*, como lo evidencia el buen equilibrio entre el error de entrenamiento y validación.

En contraste, arquitecturas más profundas, como la de cuatro capas ocultas, o más angostas, como la de una o dos capas, mostraron menores niveles de *accuracy* y mayores *loss* de validación. En particular, la arquitectura más simple ([784, 100, 49]) tuvo un rendimiento significativamente inferior, con un *accuracy* de validación de 0.52 y una pérdida de 1.95, lo que indica una menor capacidad para capturar la complejidad del problema.

Estos resultados sugieren que aumentar tanto la profundidad como el ancho de las capas ocultas puede mejorar el rendimiento, siempre que se mantenga un balance adecuado para evitar el *overfitting*.

De esta forma, a este modelo con arquitectura [784, 500, 300, 100, 49] se le denomina **M3**. En la Figura 4 se observa un descenso sostenido en la *loss* tanto para el conjunto de

entrenamiento como para el de validación, indicando que el modelo mejora progresivamente su capacidad de ajuste. La pérdida de validación comienza a estabilizarse hacia la época 30, lo cual sugiere que el modelo se aproxima a su máxima capacidad de generalización sin caer en un sobreajuste pronunciado.

A diferencia del modelo base M0, la brecha entre las curvas de entrenamiento y validación es considerablemente menor, lo que indica una mayor capacidad de generalización. En comparación con el modelo M1, que utilizó una arquitectura más simple y fue optimizado con Adam, el modelo M3 logra mejorar tanto en *accuracy* (0.65 vs. 0.60) como en *loss* (1.41 vs. 1.52), manteniendo un bajo tiempo de entrenamiento. Asimismo, respecto al modelo M2, que replica la configuración de M1 en *PyTorch*, M3 también obtiene mejores métricas finales en validación, demostrando que la optimización de la arquitectura contribuye significativamente al desempeño.

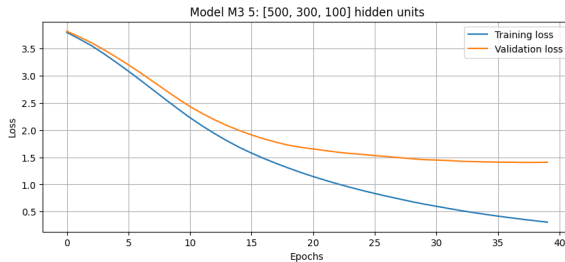


Figura 4: Evolución de la función de *loss* durante el entrenamiento del modelo M3.

En resumen, el modelo M3 logra un equilibrio superior entre capacidad de representación y generalización, superando el desempeño de los modelos M0, M1 y M2, tanto en términos de métricas como de eficiencia computacional.

Con el objetivo de explorar el fenómeno de sobreajuste (*overfitting*), se diseñó el modelo M4 utilizando una arquitectura compleja: tres capas ocultas con 1024 neuronas cada una, es decir, una red profunda y densa con una gran cantidad de parámetros entrenables.

Esta configuración fue intencionalmente elegida para evaluar cómo una alta capacidad de representación puede llevar al modelo a memorizar los datos de entrenamiento.

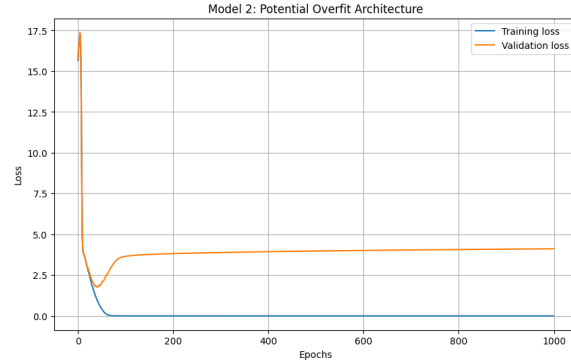


Figura 5: Evolución de la función de *loss* durante el entrenamiento del modelo M4.

Como se muestra en la Figura 5, la pérdida de entrenamiento (*training loss*) cae rápidamente a cero y se mantiene constante a lo largo de las épocas, alcanzando un *accuracy* de entrenamiento de 1. Esto indica que el modelo ha memorizado completamente los datos de entrenamiento, sin cometer errores. Sin embargo, esta capacidad de memorización no se traduce en un buen desempeño sobre el conjunto de validación.

El modelo muestra un *accuracy* de validación de 0.63 y una *loss* de validación considerablemente alta (4.11), generando una brecha significativa entre entrenamiento y validación, tanto en *accuracy* (*accuracy gap* de 0.37) como en *loss* (*loss gap* de 4.11). Estos síntomas son característicos del *overfitting*, donde el modelo ajusta excesivamente sus parámetros a las particularidades del conjunto de entrenamiento y no generaliza adecuadamente a nuevos datos.

En este caso, el sobreajuste puede atribuirse principalmente a la sobrecapacidad del modelo, es decir, al uso de una arquitectura demasiado grande en relación con la complejidad del problema y la cantidad de datos disponibles. Además, podría estar contribuyendo la

ausencia o ineficiencia de técnicas de regularización, como *dropout*, normalización por lotes (*batch normalization*) o penalizaciones de norma ($L1/L2$).

Para mitigar este problema, se recomienda reducir la complejidad del modelo (por ejemplo, disminuyendo el número de neuronas o capas), incorporar técnicas de regularización más agresivas y/o recolectar más datos de entrenamiento para aprovechar la capacidad del modelo sin incurrir en sobreajuste.

Además del modelo M4, se exploraron otras arquitecturas con el objetivo de evaluar diferentes configuraciones de profundidad y capacidad. Por ejemplo, una red con dos capas ocultas de 512 y 256 neuronas, respectivamente, alcanzó un *accuracy* de validación ligeramente superior a 0.69, pero con una *loss* de validación elevada (2.38) y una brecha de *accuracy* de 0.30. Este comportamiento también indica sobreajuste, aunque menos pronunciado que en M4.

Por otro lado, se investigó una arquitectura extremadamente profunda y ancha, compuesta por seis capas ocultas de 4096 neuronas. Sin embargo, este modelo presentó un rendimiento drásticamente inferior, con un *accuracy* cercana al azar (0.024) y una *loss* superior a 17 tanto en entrenamiento como en validación. Esto sugiere que la red no logró converger adecuadamente, posiblemente debido a dificultades numéricas o desbordes en el cálculo de gradientes (*exploding gradients*).

Debido a estos resultados, dichas arquitecturas fueron descartadas en favor de configuraciones más adecuadas para generar *overfitting*.

Por último, resulta fundamental evaluar el rendimiento de los modelos sobre un conjunto de prueba independiente, el cual no fue utilizado en ninguna etapa del entrenamiento ni de la validación. Esta evaluación permite obtener una estimación imparcial de la capacidad de generalización de cada modelo a datos completamente no vistos.

En el Cuadro 3 se presentan las métricas

obtenidas por los modelos M0 a M4 sobre el conjunto de test. Se observa que el modelo M3, que ya había demostrado el mejor desempeño en validación (con una *accuracy* de 0.65 y una *loss* de 1.41), mantiene un buen comportamiento en test, alcanzando una *accuracy* de 0.60 y una *loss* de 1.71. Esta consistencia entre validación y test sugiere que M3 posee una adecuada capacidad de generalización, evitando tanto el *overfitting* como el *underfitting*.

Modelo	Acc.	Loss	Tiempo (min)
M3	0.60	1.71	0.10
M2	0.58	1.74	0.06
M1	0.58	1.76	0.14
M0	0.58	1.79	22.97
M4	0.69	2.21	11.21

Cuadro 3: Resultados de los modelos M0, M1, M2, M3 y M4 en el conjunto de test.

En contraste, el modelo M4, a pesar de alcanzar la mayor *accuracy* en el conjunto de test (0.69), también presenta la *loss* más elevada (2.21). Este comportamiento sugiere que el modelo realiza predicciones con una alta certeza, incluso cuando estas son incorrectas, lo cual se traduce en penalizaciones más severas por parte de la función de pérdida. Esto es coherente con el *overfitting* previamente observado, donde el modelo memoriza los datos de entrenamiento pero no generaliza bien a datos nuevos. En consecuencia, se refuerza la idea de que una *accuracy* elevada, considerada de forma aislada, no garantiza un mejor rendimiento general del modelo.

Este fenómeno también puede observarse visualmente en las matrices de confusión correspondientes a cada modelo sobre el conjunto de test, disponibles en la Sección A.6 del Apéndice. Dichas matrices permiten identificar las clases en las que cada modelo tiene mayor o menor precisión, brindando una perspectiva más granular sobre su desempeño.

Por otro lado, los modelos M0, M1 y M2

obtuvieron resultados similares en test, con una *accuracy* de 0.58 y pérdidas ligeramente mayores que M3, lo que los posiciona como soluciones razonables, aunque menos efectivas.

En conjunto, estos resultados permiten concluir que M3 logra el mejor balance entre rendimiento, estabilidad y eficiencia computacional.

Una vez determinado que el modelo M3 era el que presentaba mejor rendimiento general, se lo utilizó para predecir las probabilidades *a posteriori* de pertenencia a cada una de las 49 clases para el conjunto de datos provisto en `X_COMP.npy`.

Para ello, se cargaron los datos con `numpy` y se utilizó la función `predict_best_model`, la cual permite seleccionar y aplicar el modelo correspondiente en función de su nombre. En el caso de modelos implementados con `PyTorch` (como M2, M3 y M4), se desactivó el modo entrenamiento mediante `.eval()` y se deshabilitó el cálculo de gradientes con `torch.no_grad()`, con el objetivo de optimizar la inferencia. A las salidas crudas del modelo (*logits*) se les aplicó la función `softmax` para convertirlas en probabilidades normalizadas. El resultado fue una matriz en la que cada fila corresponde a una muestra y cada columna a una clase, representando las probabilidades *a posteriori* de que cada instancia pertenezca a cada categoría del conjunto de clases.

Estas probabilidades se almacenaron en un *DataFrame* y se exportaron en formato `.csv` bajo el nombre `Basso_Ignacio_predicciones.csv`, cumpliendo con el formato solicitado: una fila por muestra y 49 columnas correspondientes a las clases del problema.

4. Conclusión

En este trabajo se implementaron y compararon distintas arquitecturas de redes neuronales multicapa para la clasificación de caracteres japoneses a partir de imágenes en escala de grises. Se comenzó con una red base (M0), entrenada mediante *backpropagation* y descenso por gradiente, y se fueron incorporando sucesivamente mejoras como *rate scheduling*, *mini-batch SGD*, regularización L2, *early stopping*, *dropout* y el optimizador Adam.

A través de la exploración sistemática de hiperparámetros y arquitecturas, se identificó que el modelo M3, con tres capas ocultas de 500, 300 y 100 unidades, y entrenado con Adam, obtuvo la mejor capacidad de generalización. Este modelo alcanzó un *accuracy* de 0.65 sobre el conjunto de validación y de 0.60 sobre el conjunto de test, superando al modelo base en términos de precisión y estabilidad.

Además, se evaluaron arquitecturas con mayor capacidad para inducir *overfitting* (modelo M4), así como configuraciones con menor complejidad que mostraron dificultades de convergencia o baja capacidad de representación. Finalmente, el modelo seleccionado se utilizó para estimar las probabilidades *a posteriori* del conjunto `X_COMP.npy`, generando una salida en el formato requerido.

Este estudio evidencia cómo la incorporación progresiva de técnicas modernas de entrenamiento y regularización puede mejorar significativamente el rendimiento de una red neuronal, demostrando la importancia de una adecuada elección de arquitectura y estrategias de optimización en tareas de clasificación complejas.

A. Apéndice

A.1. Combinaciones de la segunda etapa de búsqueda en la selección de hiperparámetros

Cuadro 4: Combinaciones de mejoras evaluadas en la segunda etapa de búsqueda.

Mejoras	Arquitectura	Hiperparámetros
Adam	[784,100,49]	lr=0.001, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon = 10^{-8}$
Adam + Exp.	[784,100,80,49]	lr=0.001, $\beta_1=0.9$, $\beta_2=0.999$, sched=exp
Mini-batch	[784,100,49]	lr=0.01, batch=64
Mini-batch	[784,100,80,49]	lr=0.01, batch=64
Adam	[784,100,80,49]	lr=0.001, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon = 10^{-8}$
Adam + Exp.	[784,100,49]	lr=0.001, $\beta_1=0.9$, $\beta_2=0.999$, sched=exp
Mini-batch	[784,200,100,80,49]	lr=0.01, batch=64

A.2. Matriz de confusión del modelo base (M0) en validation

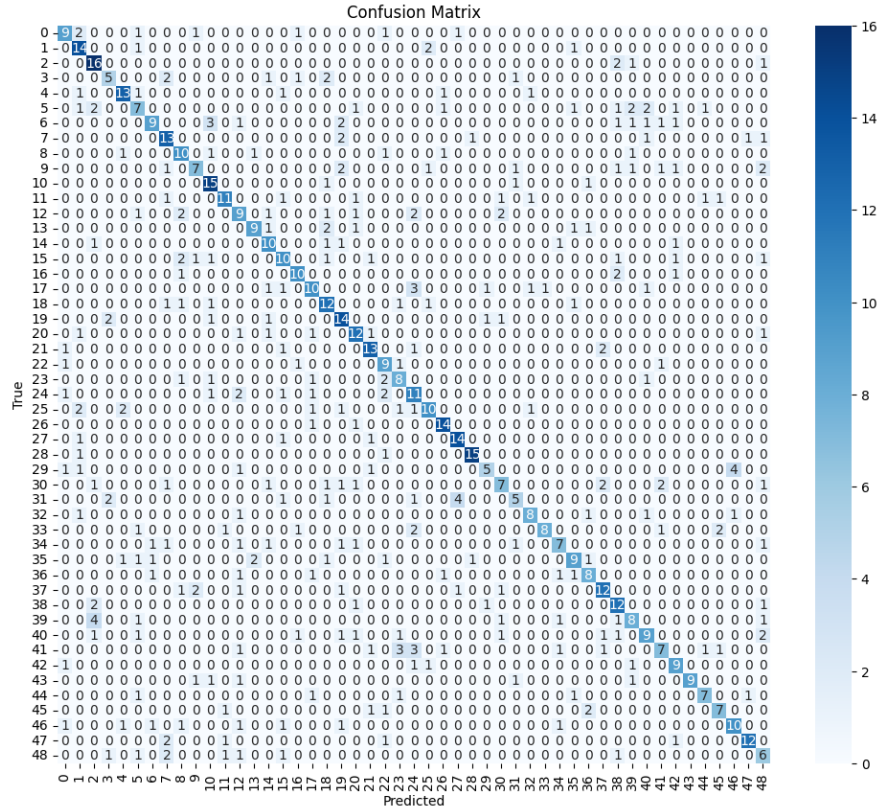


Figura 6: Matriz de confusión del modelo base (M0) en el conjunto de validación.

A.3. Métricas de performance de las combinaciones de mejoras

Cuadro 5: Combinaciones de técnicas evaluadas en la segunda etapa de búsqueda.

Mejoras	Arquitectura	Hiperparámetros	Train		Validation		Tiempo (min)
			Acc.	Loss	Acc.	Loss	
Adam	[784,100,49]	lr=0.001, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon = 10^{-8}$	0.85	0.75	0.60	1.52	0.13
Adam + Exp.	[784,100,80,49]	lr=0.001, $\beta_1=0.9$, $\beta_2=0.999$, sched=exp	0.91	0.49	0.61	1.54	0.19
Mini-batch	[784,100,49]	lr=0.01, batch=64	0.77	0.97	0.60	1.55	0.32
Mini-batch	[784,100,80,49]	lr=0.01, batch=64	0.87	0.64	0.60	1.56	0.33
Adam	[784,100,80,49]	lr=0.001, $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon = 10^{-8}$	0.93	0.43	0.62	1.57	0.14
Adam + Exp.	[784,100,49]	lr=0.001, $\beta_1=0.9$, $\beta_2=0.999$, sched=exp	0.85	0.75	0.61	1.57	0.13
Mini-batch	[784,200,100,80,49]	lr=0.01, batch=64	0.98	0.22	0.61	1.61	0.64

A.4. Matriz de confusión del modelo M1 en validation

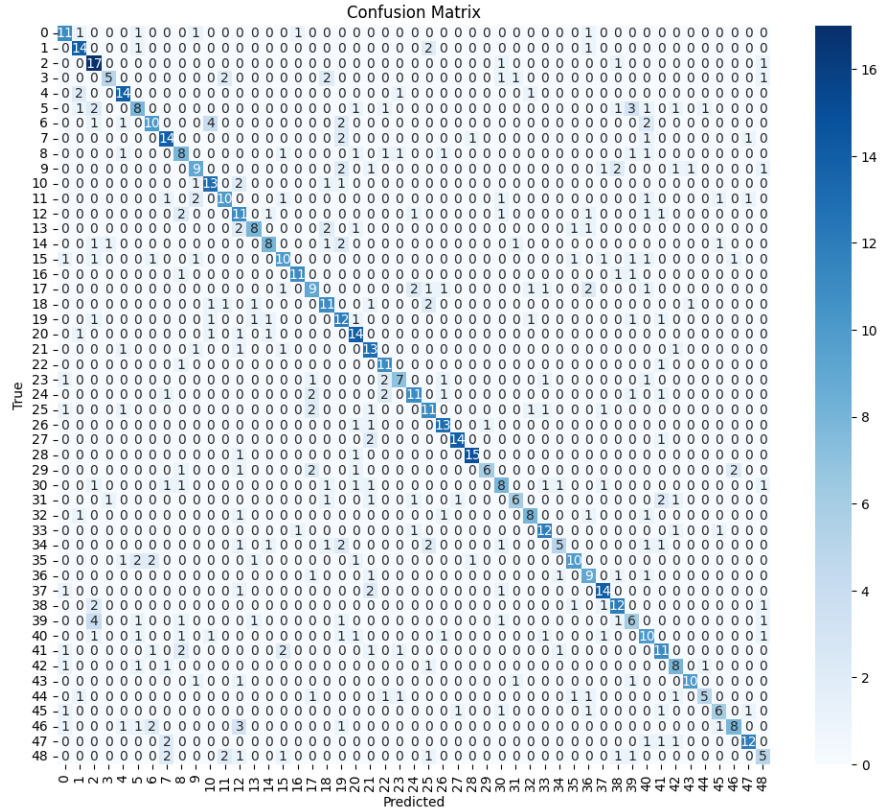


Figura 7: Matriz de confusión del modelo M1 en el conjunto de validación.

A.5. Arquitecturas evaluadas en sobre M2

Cuadro 6: Desempeño de distintas arquitecturas evaluadas.

Arquitectura	# Capas ocultas	Unidades ocultas	Train Acc.	Val. Acc.	Val. Loss
[784, 500, 300, 100, 49]	3	900	0.95	0.65	1.41
[784, 200, 200, 200, 49]	3	600	0.87	0.62	1.44
[784, 300, 150, 50, 49]	3	500	0.79	0.57	1.72
[784, 200, 100, 49]	2	300	0.71	0.56	1.72
[784, 400, 200, 100, 50, 49]	4	750	0.87	0.58	1.72
[784, 100, 49]	1	100	0.61	0.52	1.95

A.6. Matrices de confusión de los modelos en test

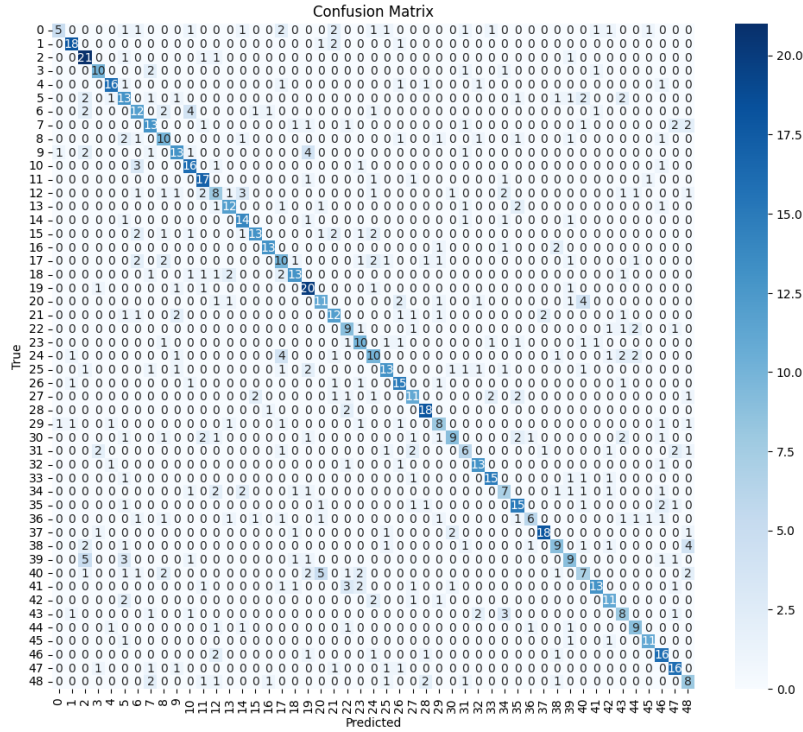


Figura 8: Matriz de confusión del modelo M0 en el conjunto de test.

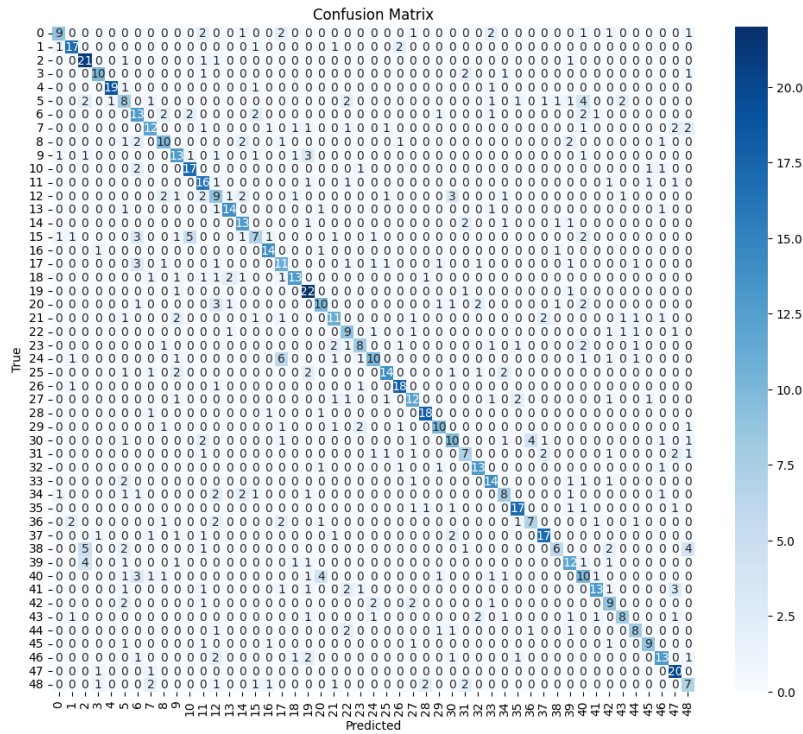


Figura 9: Matriz de confusión del modelo M1 en el conjunto de test.

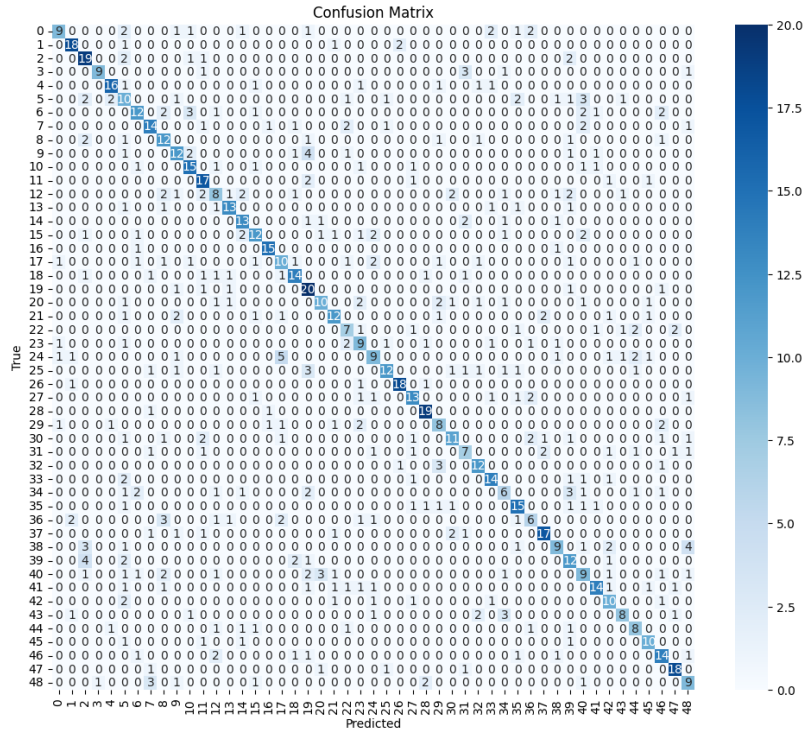


Figura 10: Matriz de confusión del modelo M2 en el conjunto de test.

